

Advanced Methods in Deep Learning

Lecture 4: Convolutional Neural Networks (CNNs)

Alexander Quispe Rojas

Universidad del Pacífico – MECA

aw.quispero@up.edu.pe aquisper@caltech.edu

April 25, 2026

Today's Roadmap

Recap and Motivation

Inductive Biases for Images

The Convolution Operation

Convolutional Layers

Pooling

Classic Architectures

Applications in Economics

PyTorch Implementation

Summary and Next Steps

Recap & Motivation

Why do we need CNNs?

Recap from Lecture 3

What we learned:

- **Adam** optimizer: momentum + adaptive LR + bias correction.
- **Initialization** matters: He (ReLU), Xavier (sigmoid/tanh).
- **Regularization**: L1 (sparsity), L2 (weight decay), dropout.
- **Batch normalization**: normalize activations per batch.
- **UAT**: depth is exponentially more efficient than width.

Today's question:

How do we apply neural networks to **images** and other spatial data?

Answer: Convolutional Neural Networks (CNNs) — networks that exploit spatial structure.

The Problem with Fully-Connected Networks for Images

Consider a 100×100 grayscale image flattened to a vector of 10,000 pixels.

A fully-connected layer with H hidden units has $10,000 \times H$ weights:

Hidden units H	Weights (input layer only)
1	10,000
100	1,000,000
1,000	10,000,000
10,000	100,000,000
100,000	1,000,000,000 (1 billion!)

And this is just one layer of a small grayscale image.

Real images: $224 \times 224 \times 3 = 150,528$ inputs. Real datasets (ImageNet): 1.2M images.

Parameter explosion + insufficient data $\Rightarrow$ severe overfitting.

Two Components to Solve Any Task

priors

knowledge
assumed

learning

knowledge
extracted

Strong priors, minimal learning → fast/easy, but **rigid**.

Weak priors, much learning → flexible, but **slow/data-hungry**.

For images, we need to add the right **inductive biases** (priors) to make learning tractable.

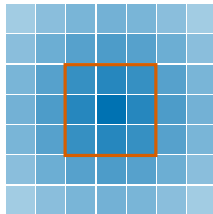
Inductive Biases for Images

Locality: Nearby Pixels Share Patterns

Observation: structure in images is **local**.

- Nearby **pixels** are correlated (sky, grass, skin all show smooth color gradients).
- Nearby **patches** form features (edges, textures).
- Nearby **regions** form parts (eyes, mouths, wheels).

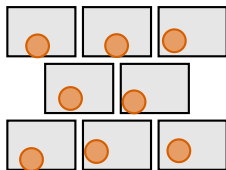
Inductive bias: a feature at one location should depend **only on its local neighborhood**, not on distant pixels.



A 3×3 neighborhood (red box) captures the local structure.

Translation Invariance: Position-Independent Features

Observation: the identity of an object does not depend on its absolute position.



Example: a cat is a cat whether it appears in the top-left or bottom-right of the image.

Inductive bias: the same **feature detector** should be applied at every spatial location $\Rightarrow$ **weight sharing across positions**.

From Inductive Biases to CNNs

Two key inductive biases for images:

Property	Architectural implementation
Locality	Each output depends on a small spatial region of the input
Translation invariance	The same filter weights are reused at every location

Together: Convolutional Neural Networks (CNNs)

Consequences:

- Number of parameters becomes **independent of input size**.
- The model generalizes much better with limited data.
- Computationally efficient (parallelizable on GPUs).

The Convolution Operation

1D Convolution: Mathematical Definition

1D Convolution

Given input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{K} \in \mathbb{R}^k$, the (cross-)convolution is:

$$(\mathbf{x} * \mathbf{K})[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j] \cdot \mathbf{K}[j]$$

Note: in DL, “convolution” usually means cross-correlation (no kernel flipping).

Intuition: slide the kernel across the input, computing dot products at each position.

Example: input $\mathbf{x} = (1, 2, 3, 4, 5)$, kernel $\mathbf{K} = (1, 0, -1)$:

$$\text{output}[0] = 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) = -2$$

$$\text{output}[1] = 2 \cdot 1 + 3 \cdot 0 + 4 \cdot (-1) = -2$$

$$\text{output}[2] = 3 \cdot 1 + 4 \cdot 0 + 5 \cdot (-1) = -2$$

2D Convolution: Numerical Example

Input image 5×5 and kernel 3×3 :

	3	3	2	1	0
	0	0	1	3	1
Input	3	1	2	2	3
	2	0	0	2	2
	2	0	0	0	1

	0	1	2
	2	2	0
K	0	1	2

12	12	17
10	17	19
9	6	14

Output[0,0] (red region)

$$= 3 \cdot 0 + 3 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 2 + 1 \cdot 0 + 3 \cdot 0 + 1 \cdot 1 + 2 \cdot 2 = 12$$

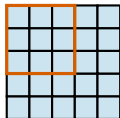
Slide the kernel one position $\rightarrow$ compute next output, etc.

Stride and Padding

Stride S : how many pixels the kernel moves per step. **Padding P :** zeros around the border.

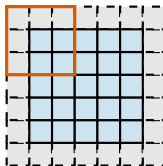
Stride = 1, no padding

Output smaller than input



Stride = 1, padding = 1 ("same")

Output same size as input



Output Size Formula

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

W_{in} : input width F : filter (kernel) size P : padding S : stride

Output Size: Worked Examples

Formula: $W_{\text{out}} = \lfloor (W_{\text{in}} + 2P - F) / S \rfloor + 1$

W_{in}	Filter F	Padding P	Stride S	Computation	W_{out}
28	3	0	1	$\lfloor (28 + 0 - 3) / 1 \rfloor + 1$	26
28	3	1	1	$\lfloor (28 + 2 - 3) / 1 \rfloor + 1$	28
28	5	2	1	$\lfloor (28 + 4 - 5) / 1 \rfloor + 1$	28
224	3	1	2	$\lfloor (224 + 2 - 3) / 2 \rfloor + 1$	112
224	7	3	2	$\lfloor (224 + 6 - 7) / 2 \rfloor + 1$	112

Common patterns:

- “Same” convolution ($P = (F - 1) / 2$, $S = 1$): preserves size. Useful for stacking layers.
- Strided convolution ($S = 2$): halves spatial size. Used for downsampling.

Filters as Feature Detectors

Different kernels detect different features:

Sobel (vertical edges)

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Detects vertical changes in intensity.

Gaussian (blur)

$$\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Smooths the image.

Sharpen

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

Enhances edges.

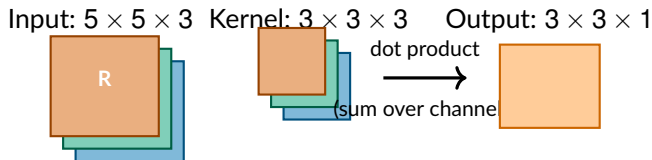
Key insight: in classical computer vision, these filters were **hand-designed**.

In CNNs, the filters are **learned** from data via backpropagation.

Convolutional Layers

Multiple Input Channels (RGB Images)

A color image has **3 channels** (R, G, B). The kernel must also have 3 channels.



Math: for input with C_{in} channels and a $F \times F \times C_{in}$ kernel:

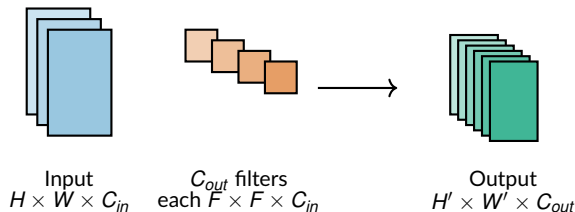
$$\text{output}[i, j] = \sum_{c=1}^{C_{in}} \sum_{u=0}^{F-1} \sum_{v=0}^{F-1} \text{input}[i+u, j+v, c] \cdot \text{kernel}[u, v, c] + b$$

Each filter “sees” all input channels but produces a **single** output channel.

Multiple Filters $\rightarrow$ Multiple Output Channels

One filter produces one output channel (feature map).

Multiple filters produce multiple output channels.



Total parameters for one conv layer:

$$\underbrace{F \times F \times C_{in} \times C_{out}}_{\text{weights}} + \underbrace{C_{out}}_{\text{biases}}$$

Parameter Comparison: FC vs Conv

Setting: input $224 \times 224 \times 3$, output 64 feature maps.

Layer type	Parameter count	Total
Fully-connected	$224 \times 224 \times 3 \times 64 =$	9,633,792
Conv (3×3)	$3 \times 3 \times 3 \times 64 + 64 =$	1,792
Conv (5×5)	$5 \times 5 \times 3 \times 64 + 64 =$	4,864
Conv (7×7)	$7 \times 7 \times 3 \times 64 + 64 =$	9,472

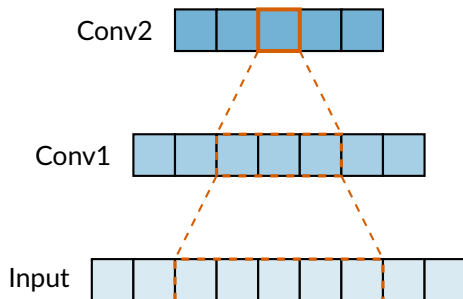
CNN uses $\approx 5,000\times$ fewer parameters than FC for the same task.

Why?

- **Locality:** each output uses only a small input region.
- **Weight sharing:** the same kernel is reused at every spatial location.

Receptive Field

Definition: the receptive field of a neuron is the region of the **input image** that affects its output.



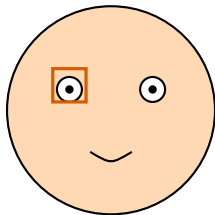
Each layer uses a 3×3 kernel (1D simplification: 1×3):

- Layer 1 neuron sees **3** input pixels.
- Layer 2 neuron sees **5** input pixels.
- Layer L neuron sees $2L + 1$ input pixels.

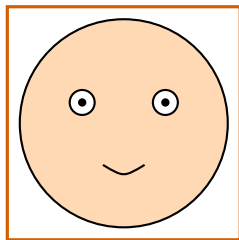
Receptive field grows linearly with depth $\Rightarrow$ deep CNNs “see” the whole image.

Why Does Receptive Field Matter?

Key principle: a neuron's output depends **only** on the pixels inside its receptive field. Anything outside is invisible to it.



Small RF (early layer)
sees only an eye
can't classify the face



Large RF (deep layer)
sees the whole face
can classify it

Practical rule: for a task that requires recognizing an object of size $S \times S$, you need a network with $\text{RF} \geq S \times S$.

This is the main reason CNNs use **many layers** and **pooling**: to grow RF.

Receptive Field: The Math

Two recurrences track the receptive field through the network:

Receptive Field Recurrence

$$j_l = j_{l-1} \cdot S_l$$

(jump = cumulative stride at layer l)

$$\text{RF}_l = \text{RF}_{l-1} + (F_l - 1) \cdot j_{l-1}$$

(receptive field at layer l)

Initial conditions: $j_0 = 1$, $\text{RF}_0 = 1$.

Intuition:

- j_l = how many input pixels you “jump” when moving by 1 pixel in layer l .
- Each new layer adds $(F_l - 1)$ more pixels to the RF, but each pixel is now j_{l-1} input pixels apart.
- **Stride / pooling layers don't add much to RF directly, but they double the jump** for all subsequent layers.

Receptive Field: Step-by-Step Calculation

Apply the recurrence to our 7-layer CNN:

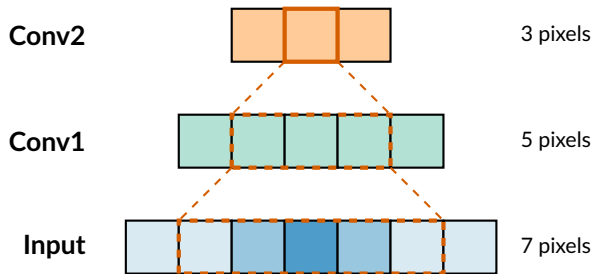
l	Operation	F_l	S_l	$j_l = j_{l-1} S_l$	$RF_l = RF_{l-1} + (F_l - 1)j_{l-1}$	Result
0	Input	—	—	1	1	$RF_0 = 1$
1	Conv 3×3	3	1	$1 \cdot 1 = 1$	$1 + 2 \cdot 1$	$RF_1 = 3$
2	Conv 3×3	3	1	$1 \cdot 1 = 1$	$3 + 2 \cdot 1$	$RF_2 = 5$
3	MaxPool 2×2	2	2	$1 \cdot 2 = 2$	$5 + 1 \cdot 1$	$RF_3 = 6$
4	Conv 3×3	3	1	$2 \cdot 1 = 2$	$6 + 2 \cdot 2$	$RF_4 = 10$
5	Conv 3×3	3	1	$2 \cdot 1 = 2$	$10 + 2 \cdot 2$	$RF_5 = 14$
6	MaxPool 2×2	2	2	$2 \cdot 2 = 4$	$14 + 1 \cdot 2$	$RF_6 = 16$
7	Conv 3×3	3	1	$4 \cdot 1 = 4$	$16 + 2 \cdot 4$	$RF_7 = 24$

Watch the jump: j goes $1 \rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 2 \rightarrow 2 \rightarrow 4 \rightarrow 4$. Each pooling layer **doubles** it, so subsequent convs add *twice as much* to the RF.

This is why deeper networks “see” the whole image **exponentially** faster with pooling.

Tracing Pixels: 1D Walkthrough

Let's trace which input pixels affect a single output neuron. Apply two 1×3 convs (stride 1, no padding) to a 1D input of 7 pixels.

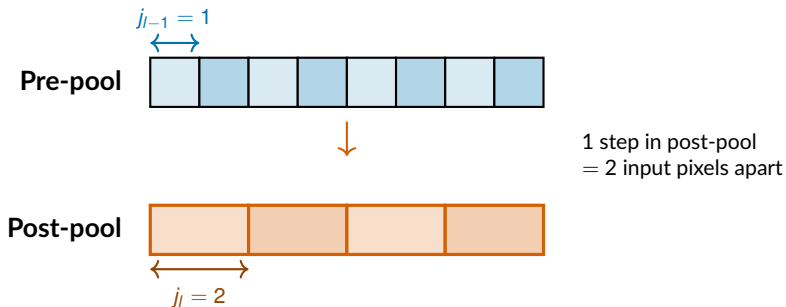


Verify with the formula:

- Layer 1: $RF_1 = 1 + (3 - 1) \cdot 1 = 3$ ✓ (red box at Conv1 spans 3 cells).
- Layer 2: $RF_2 = 3 + (3 - 1) \cdot 1 = 5$ ✓ (red box at Input spans 5 cells).

Visualizing the Jump After Pooling

Key idea: after a pooling layer with stride 2, each pixel in the next layer corresponds to **2 input pixels**, not 1.



Consequence: when the next 3×3 conv slides over the post-pool layer, those 3 “adjacent” positions are actually **6 input pixels apart!**

That’s why the formula has $(F - 1) \cdot j_{l-1}$ — each kernel step covers j_{l-1} input pixels.

The jump factor j explains the multiplier on receptive field growth.

With vs Without Pooling: RF Comparison

Same network architecture, but compare RF growth **with** and **without** pooling:

Layer	1	2	3	4	5	6	7
Operation (no pool)	Conv	Conv	Conv	Conv	Conv	Conv	Conv
RF (no pool)	3	5	7	9	11	13	15
Operation (with pool)	Conv	Conv	Pool	Conv	Conv	Pool	Conv
RF (with pool)	3	5	6	10	14	16	24

Without pooling: RF grows **linearly**, $RF_l = 2l + 1$. Need many layers to “see” a large area.

With pooling: RF grows **multiplicatively**. Each pool doubles future increments.

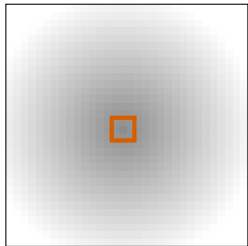
To cover a 32×32 image:

- Without pooling: need $L = 16$ conv layers ($RF = 33$).
- With pooling: need only ~ 7 layers (as shown).

Pooling makes deep networks practical.

What Does Each Layer “See”?

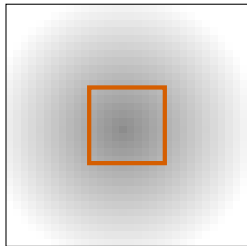
The receptive field tells us **how much of the input image** a neuron at a given layer “sees”.



Layer 1

RF = 3×3

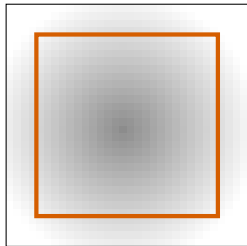
edges



Layer 4

RF = 10×10

parts



Layer 7

RF = 24×24

full object

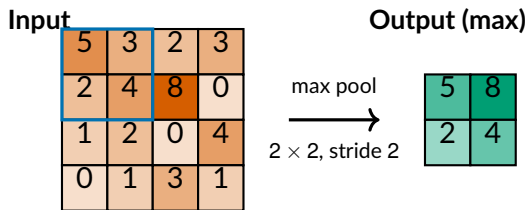
Hierarchical features: early layers detect **edges**, middle layers detect **parts** (an eye, a wheel), deep layers recognize **full objects/scenes**.

Depth matters: each layer **composes** features from the previous one.

Pooling

Pooling: Aggregation and Downsampling

Pooling aggregates values in local regions of a feature map.



Two main types:

- **Max pooling:** take the maximum value in each window. Most common.
- **Average pooling:** take the average. Sometimes used in final layers.

Why pool? (1) **Downsampling** (reduces computation and parameters); (2) **Translation invariance** (small shifts don't change max).

Pooling vs Strided Convolutions

Modern alternative: use **strided convolutions** ($S = 2$) instead of pooling.

	Pooling	Strided Convolution
Operation	Max / Average (no params)	Learnable filter with $S = 2$
Parameters	0	$F \times F \times C_{in} \times C_{out}$
Used in	LeNet, AlexNet, VGG	ResNet, modern architectures

Trend: modern architectures prefer strided convs because the network **learns the optimal downsampling**.

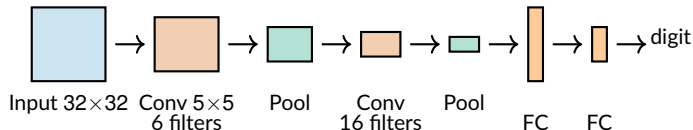
Pop quiz: a $5 \times 5 \times 16$ input with 36 filters of 3×3 (stride 1, no padding):

- Each filter: $3 \times 3 \times 16$ (matches input channels).
- Output: $3 \times 3 \times 36$.

Classic CNN Architectures

LeNet-5 (LeCun et al., 1998)

First successful CNN. Designed for handwritten digit recognition (MNIST).



Key innovations:

- Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ FC pattern (still standard today).
- Tanh activation, average pooling.
- About **60,000 parameters**.

Limitation: too small for natural images. Required compute didn't exist for larger versions.

AlexNet (Krizhevsky et al., 2012)

The ImageNet revolution. Won ILSVRC 2012 by a huge margin (top-5 error: 16% vs 26%).

Architecture: 5 conv layers + 3 FC layers, $\sim$ **60 million parameters**.

Key innovations:

- **ReLU** activations (faster training than tanh/sigmoid).
- **Dropout** for regularization.
- Trained on **2 GPUs** (model parallelism due to memory limits).
- **Data augmentation** (random crops, color jittering).
- Trained on **ImageNet** (1.2M images, 1000 classes).

This paper kickstarted the modern deep learning era.

VGG (Simonyan & Zisserman, 2014)

Idea: use **small** 3×3 filters but stack **many** layers.

VGG-16 architecture: 13 conv layers + 3 FC layers, **138 million parameters**.

Why 3×3 filters? Stacking two 3×3 convs has the same receptive field as one 5×5 conv, but with:

- **Fewer parameters:** $2 \cdot 3^2 = 18$ vs $5^2 = 25$.
- **More non-linearities** (two ReLUs vs one).
- **Better expressivity** for the same receptive field.

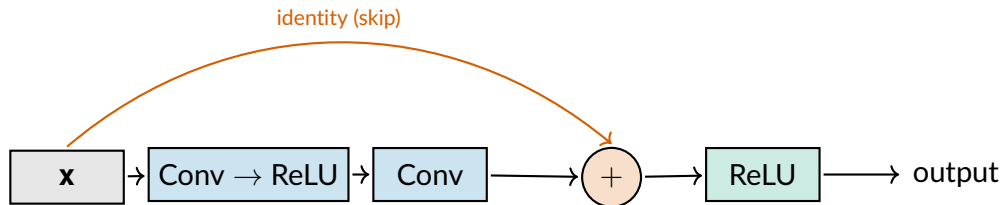
Key Insight

The VGG insight: **depth** matters more than filter size. Use uniform 3×3 convolutions and let depth do the work.

ResNet (He et al., 2015)

The degradation problem: as networks get deeper (50+ layers), training loss **increases**, not just test loss.

Solution: residual / skip connections.



Mathematical formulation: $\mathbf{h}^{(l+1)} = \mathcal{F}(\mathbf{h}^{(l)}; \mathbf{W}^{(l)}) + \mathbf{h}^{(l)}$

Why this works:

- Gradient flow: $\nabla \mathbf{h}^{(l)}$ has a direct path back via the identity (no vanishing gradient).
- If the optimal layer is identity, the network can easily learn $\mathcal{F} \approx 0$.
- Allows training networks with **152, 1000+ layers**.

Architecture Evolution: A Visual Summary

Architecture	Year	Layers	Params	Key idea
LeNet-5	1998	7	60K	First successful CNN
AlexNet	2012	8	60M	ReLU + Dropout + GPU
VGG-16	2014	16	138M	Uniform 3×3 filters
GoogLeNet	2014	22	6.8M	Inception modules
ResNet-152	2015	152	60M	Skip connections
DenseNet	2017	121+	8M	Dense skip connections
EfficientNet	2019	varies	5–66M	Compound scaling

General trends: deeper networks · smaller filters (3×3 standard) · skip connections · better parameter efficiency.

Applications in Economics

Predicting Poverty from Satellite Imagery

Landmark Paper

Jean, Burke, Xie, Davis, Lobell, Ermon (2016). Combining Satellite Imagery and Machine Learning to Predict Poverty. *Science*, 353(6301), 790–794.

Problem: household survey data is scarce in many African countries. Can we predict consumption from satellite imagery?

Pipeline:

1. Take a **pretrained CNN** (VGG on ImageNet).
2. Fine-tune on satellite imagery to predict **nightlight intensity** (proxy for activity).
3. Use learned features to predict **household consumption** from daytime images.

Result: explained up to **75% of variation** in local economic outcomes across 5 African countries.

This is **transfer learning** — covered more in Lecture 9.

Other Economic Applications of CNNs

Land use and agricultural monitoring:

- **Crop yield prediction** from satellite imagery (You, Li, Low, Lobell, Ermon, 2017).
- **Deforestation tracking** for environmental economics.
- **Land cover classification** (USDA, ESA).

Real estate and urban economics:

- **Property valuation** from street view images (Naik et al., 2017).
- **Neighborhood characteristics** from urban imagery (Glaeser, Kominers et al., 2018).
- **Gentrification detection** from time-series imagery.

Development economics:

- **Wealth indices** from satellite + survey data (Yeh et al., 2020).
- **Health outcomes** from environmental imagery.
- **Conflict and disaster damage** from before/after imagery.

Why CNNs Matter for Economists

Three reasons:

1. **Unstructured data is everywhere.** Satellite imagery, street views, scanned documents, X-rays in health economics, etc. CNNs unlock these for econometric analysis.
2. **Filling data gaps.** In developing countries, traditional surveys are sparse. CNNs on satellite data can fill gaps at **high spatial resolution** and **low cost**.
3. **Causal inference + DL.** CNNs can be combined with double machine learning, instrumental variables, and other causal frameworks to estimate treatment effects in non-tabular settings.

Caveat: interpretability and identification remain challenges. CNNs are excellent at *prediction*, but *causal inference* requires careful design.

PyTorch Implementation

Building a Simple CNN in PyTorch

```
1 import torch
2 import torch.nn as nn
3
4 class SimpleCNN(nn.Module):
5     def __init__(self, num_classes=10):
6         super().__init__()
7         # Input: 3 x 32 x 32 (CIFAR-10)
8         self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
9         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
10        self.pool = nn.MaxPool2d(2, 2)
11        self.fc1 = nn.Linear(64 * 8 * 8, 128)
12        self.fc2 = nn.Linear(128, num_classes)
13
14    def forward(self, x):
15        x = self.pool(torch.relu(self.conv1(x))) # 32 x 16 x 16
16        x = self.pool(torch.relu(self.conv2(x))) # 64 x 8 x 8
17        x = x.flatten(1) # 64*8*8
18        x = torch.relu(self.fc1(x))
19        return self.fc2(x)
```

Using a Pretrained ResNet

Transfer learning: use a network pretrained on ImageNet, fine-tune for your task.

```
1 import torch
2 import torchvision.models as models
3 import torch.nn as nn
4
5 # Load pretrained ResNet-50
6 model = models.resnet50(weights='IMAGENET1K_V2')
7
8 # Freeze all parameters
9 for param in model.parameters():
10     param.requires_grad = False
11
12 # Replace the final FC layer for our task (e.g., 5 classes)
13 model.fc = nn.Linear(model.fc.in_features, 5)
14
15 # Only the new FC layer will be trained
16 optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

This is the technique used in Jean et al. (2016).

Summary & Next Steps

Lecture 4 Summary

1. **Inductive biases for images:** locality + translation invariance.
2. **Convolution:** slide a small kernel across the input, compute dot products. Output size: $\lfloor (W + 2P - F) / S \rfloor + 1$.
3. **Multi-channel convolutions:** kernel has the same number of channels as input; multiple kernels produce multiple output feature maps.
4. **Pooling:** aggregates feature values for downsampling and invariance.
5. **Classic architectures:** LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet (skip connections enable very deep networks).
6. **Economic applications:** satellite imagery for poverty prediction, land use classification, real estate, etc.
7. **Transfer learning:** fine-tune ImageNet-pretrained models for new tasks.

Next Lecture: Word Embeddings & RNNs

Lecture 5 (Monday, April 27):

- Word2Vec: representing words as vectors.
- Skip-gram and CBOW.
- Recurrent Neural Networks (RNNs).
- LSTM and GRU.
- Applications to text and time series.

Recommended reading:

- Goodfellow et al., *Deep Learning*, Chapter 9 (CNNs).
- He et al. (2015), *Deep Residual Learning for Image Recognition*.
- Jean et al. (2016), *Combining Satellite Imagery and ML to Predict Poverty*.

Homework 2 will be released after Lecture 5 (covers CNNs).

Questions?

References

- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS*.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition (VGG). *ICLR*.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition (ResNet). *CVPR*.
- Jean, N., Burke, M., Xie, M., Davis, W. M., Lobell, D. B., & Ermon, S. (2016). Combining satellite imagery and machine learning to predict poverty. *Science*, 353(6301).
- You, J., Li, X., Low, M., Lobell, D., & Ermon, S. (2017). Deep Gaussian process for crop yield prediction based on remote sensing data. *AAAI*.
- Yeh, C. et al. (2020). Using publicly available satellite imagery and deep learning to understand economic well-being in Africa. *Nature Communications*.